

# **Desenvolvendo sua Primeira Aplicação Web com Spring Boot MySQL**



- **Instrutor: Djalma Batista Barbosa Junior**
- **E-mail: [djalma.batista@fiemg.com.br](mailto:djalma.batista@fiemg.com.br)**

## Enunciado:

### Cenário: Gerenciador Simples de Tarefas (To-Do List)

Imagine a necessidade de organizar tarefas diárias, seja para uso pessoal ou para um pequeno time. Precisamos de um sistema simples onde possamos registrar o que precisa ser feito e marcar as tarefas como concluídas. Este será o nosso cenário: um Gerenciador Simples de Tarefas.

#### Cada tarefa em nosso sistema terá basicamente duas informações:

- Uma descrição textual do que precisa ser feito.
- Um indicador (booleano: verdadeiro ou falso) mostrando se a tarefa foi concluída ou não.

A escolha deste cenário se deve à sua simplicidade. Ele nos permite focar nos aspectos técnicos essenciais do Spring Boot, da integração com banco de dados via JPA e da criação de uma API RESTful, sem nos prendermos a regras de negócio complexas neste momento inicial.

## Objetivo da Atividade

O objetivo principal desta Situação de Aprendizagem é construir o backend (a API RESTful) para o Gerenciador Simples de Tarefas. Esta API deverá permitir que aplicações cliente (como um futuro aplicativo web ou mobile) possam realizar as seguintes operações:

- Criar tarefas.
- Listar todas as tarefas existentes.
- Buscar os detalhes de uma tarefa específica pelo seu identificador único (ID).
- Atualizar os dados de uma tarefa existente (modificar sua descrição ou marcar como concluída/não concluída).
- Excluir uma tarefa que não é mais necessária.

Todos os dados das tarefas deverão ser armazenados de forma persistente em um banco de dados MySQL. Ao concluir esta atividade, vocês terão desenvolvido uma aplicação Spring Boot funcional, compreendendo os fundamentos da criação de APIs RESTful e da interação com bancos de dados relacionais utilizando Spring Data JPA.

## Guia Descritivo das Funções (Requisitos Funcionais da API)

A API RESTful que vamos construir deve oferecer as seguintes funcionalidades, mapeadas para operações CRUD e métodos HTTP padrão:

### Criar Tarefa:

- Operação: Criar (Create)
- Método HTTP: POST
- Endpoint: /api/tarefas
- Descrição: Permite adicionar uma nova tarefa ao sistema.
- Entrada: No corpo (body) da requisição HTTP, deve ser enviado um objeto JSON contendo a descrição da nova tarefa. Exemplo: {"descricao": "Estudar Spring Boot"}. O status concluída deve ser automaticamente definido como false ao criar uma tarefa.
- Saída: Retorna um objeto JSON representando a tarefa recém-criada, incluindo o id único que foi gerado automaticamente pelo banco de dados, e um status HTTP 201 Created. Exemplo: {"id": 1, "descricao": "Estudar Spring Boot", "concluída": false}.

### Listar Todas as Tarefas:

- Operação: Ler (Read)
- Método HTTP: GET
- Endpoint: /api/tarefas
- Descrição: Permite obter uma lista de todas as tarefas cadastradas no sistema.
- Entrada: Nenhuma informação adicional é necessária.
- Saída: Retorna um array (lista) de objetos JSON, onde cada objeto representa uma tarefa cadastrada, e um status HTTP 200 OK. Exemplo: []. Se não houver tarefas, retorna uma lista vazia.

## Buscar Tarefa por ID:

- Operação: Ler (Read)
- Método HTTP: GET
- Endpoint: /api/tarefas/{id}
- Descrição: Permite obter os detalhes de uma única tarefa, identificada pelo seu id.
- Entrada: O id da tarefa desejada deve ser informado como parte da URL (path parameter). Exemplo: /api/tarefas/1.
- Saída: Se a tarefa com o id informado existir, retorna um objeto JSON com os dados da tarefa e um status HTTP 200 OK. Exemplo: {"id": 1, "descricao": "Estudar Spring Boot", "concluída": false}. Se a tarefa não for encontrada, retorna um status HTTP 404 Not Found.

## Atualizar Tarefa:

- Operação: Atualizar (Update)
- Método HTTP: PUT
- Endpoint: /api/tarefas/{id}
- Descrição: Permite modificar os dados (descrição e/ou status de conclusão) de uma tarefa existente.
- Entrada: O id da tarefa a ser atualizada deve ser informado na URL. No corpo da requisição, deve ser enviado um objeto JSON com os campos que se deseja atualizar. Exemplo: {"descricao": "Estudar Spring Boot e JPA", "concluida": true}.
- Saída: Se a tarefa com o id informado existir, ela é atualizada e a API retorna um objeto JSON com os dados atualizados da tarefa e um status HTTP 200 OK. Exemplo: {"id": 1, "descricao": "Estudar Spring Boot e JPA", "concluida": true}. Se a tarefa não for encontrada, retorna um status HTTP 404 Not Found.

## Deletar Tarefa:

- Operação: Deletar (Delete)
- Método HTTP: DELETE
- Endpoint: /api/tarefas/{id}
- Descrição: Permite remover uma tarefa do sistema.
- Entrada: O id da tarefa a ser deletada deve ser informado na URL. Exemplo: /api/tarefas/1.

- Saída: Se a tarefa com o id informado existir, ela é removida do banco de dados e a API retorna um status HTTP indicando sucesso (geralmente 200 OK ou 204 No Content, que significa sucesso sem conteúdo no corpo da resposta). Se a tarefa não for encontrada, retorna um status HTTP 404 Not Found.

•

## Exemplo Prático: Guia Passo a Passo Detalhado

Agora, vamos colocar a mão na massa! Este guia detalhado cobrirá todas as etapas necessárias para construir e testar nossa API de gerenciamento de tarefas.

### Preparando o Ambiente de Desenvolvimento

Antes de começarmos a codificar, é crucial garantir que nosso ambiente de desenvolvimento esteja corretamente configurado com todas as ferramentas necessárias.

Um ambiente consistente evita problemas futuros e garante que todos possam seguir os passos da mesma forma. Os passos a seguir focam principalmente no Windows, mas forneceremos dicas e links para macOS e Linux quando relevante.

### Instalando o Java JDK (Java Development Kit)

- Contexto: O JDK é o coração do desenvolvimento Java. Ele contém o compilador (javac), a máquina virtual Java (java - JVM) e outras ferramentas essenciais para transformar nosso código-fonte em uma aplicação executável. Aplicações Spring Boot são aplicações Java, portanto, o JDK é um pré-requisito indispensável. Recomendamos o uso de versões LTS (Long-Term Support), como JDK 17 ou 21, que oferecem estabilidade e suporte prolongado.

### Passos:

1. Download: Acesse o site oficial da Oracle para baixar o Oracle JDK (<https://www.oracle.com/java/technologies/downloads/>) ou opte por uma distribuição OpenJDK como a BellSoft Liberica JDK (<https://bell-sw.com/pages/downloads/>), recomendada pelo Spring. Escolha a versão LTS desejada (17 ou 21) e o instalador

apropriado para seu sistema operacional (para Windows, geralmente um arquivo .exe ou .msi).

2. Instalação (Windows): Execute o arquivo baixado (ex: jdk-21\_windows-x64\_bin.exe). Geralmente, é necessário ter privilégios de administrador. Siga as instruções do instalador, aceitando os termos e mantendo as opções padrão, a menos que tenha um motivo específico para alterá-las. O JDK será instalado, por padrão, em um diretório como C:\Program Files\Java\jdk-21.
3. Configuração das Variáveis de Ambiente (Windows - Crucial!): Este é um passo fundamental. O sistema operacional e outras ferramentas precisam saber onde encontrar os executáveis do Java.
  - JAVA\_HOME: Esta variável aponta para o diretório principal da instalação do JDK.
  - Pesquise por "Editar as variáveis de ambiente do sistema" na barra de pesquisa do Windows e abra.
  - Na janela "Propriedades do Sistema", clique em "Variáveis de Ambiente...".
  - Na seção "Variáveis do sistema", clique em "Novo...".
  - Nome da variável: JAVA\_HOME
  - Valor da variável: C:\Program Files\Java\jdk-21 (ajuste o caminho exato da sua instalação).
  - Clique em OK.
  - Path: Esta variável informa ao sistema onde procurar por executáveis na linha de comando. Precisamos adicionar o diretório bin do JDK ao Path.
  - Ainda na seção "Variáveis do sistema", localize a variável Path, selecione-a e clique em "Editar...".
  - Clique em "Novo" e adicione a seguinte entrada: %JAVA\_HOME%\bin. O uso de %JAVA\_HOME% torna a configuração mais flexível se você atualizar o JDK no futuro.
  - Clique em OK em todas as janelas abertas para salvar as alterações.
  - Por que isso é tão importante? A configuração correta das variáveis de ambiente é a base para que ferramentas como Maven, Gradle e sua IDE localizem e utilizem o JDK corretamente.

Se o Path não estiver configurado, comandos como java ou javac não serão reconhecidos no terminal. Se JAVA\_HOME não estiver definido, ferramentas de build podem falhar ao tentar compilar seu projeto. Erros nessas configurações podem levar a falhas inesperadas e difíceis de diagnosticar mais tarde.

4. Verificação: Abra um novo prompt de comando ou terminal (as variáveis de ambiente só são carregadas em novas sessões). Digite os seguintes comandos e verifique as saídas:

- java -version (Deve exibir a versão do Java instalada).
- javac -version (Deve exibir a versão do compilador Java).
- echo %JAVA\_HOME% (No Windows) ou echo \$JAVA\_HOME (No Linux/macOS) (Deve exibir o caminho que você configurou).
- (Para macOS/Linux): A instalação pode ser feita via gerenciadores de pacotes (como Homebrew/SDKMAN! para macOS, apt para Debian/Ubuntu, yum/dnf para CentOS/Fedora) ou baixando o arquivo.tar.gz e configurando as variáveis JAVA\_HOME e PATH no seu arquivo de perfil do shell (ex: bashrc,. zshrc, /etc/environment).

## Instalando o Apache Maven

- Contexto: Maven é uma ferramenta poderosa para gerenciar o ciclo de vida e as dependências de projetos Java. Ele automatiza tarefas como compilação, teste e empacotamento da aplicação. Além disso, ele baixa automaticamente as bibliotecas (dependências, como Spring Boot, Spring Data JPA, etc.) que nosso projeto precisa, a partir de repositórios online. Gradle é uma alternativa popular, mas usaremos Maven neste guia.

### Passos:

1. Download: Acesse a página de download do Apache Maven (<https://maven.apache.org/download.cgi>). Baixe o arquivo binário em formato .zip (para Windows) ou.tar.gz (para Linux/macOS). Exemplo: apache-maven-3.9.x-bin.zip.

2. Extração: Crie uma pasta para o Maven (ex: C:\Program Files\Maven) e extraia o conteúdo do arquivo baixado para dentro dela. Você terá uma estrutura de diretórios como C:\Program Files\Maven\apache-maven-3.9.x.
3. Configuração das Variáveis de Ambiente (Windows): Similar ao JDK, precisamos informar ao sistema onde encontrar o Maven.
  - M2\_HOME (Opcional, mas recomendado):
  - Vá novamente em "Variáveis de Ambiente...".
  - Na seção "Variáveis do sistema", clique em "Novo...".
  - Nome da variável: M2\_HOME
  - Valor da variável: C:\Program Files\Maven\apache-maven-3.9.x (ajuste o caminho exato).
  - Clique em OK. (Embora alguns mencionem que M2\_HOME não é estritamente necessário, é uma convenção útil para outras ferramentas).
  - Path:
  - Edite a variável Path novamente.
  - Clique em "Novo" e adicione: %M2\_HOME%\bin.
  - Clique em OK em todas as janelas.

### **Verificação: Abra um novo prompt de comando e digite:**

- mvn -version (Deve exibir a versão do Maven, a versão do Java sendo utilizada por ele e o diretório JAVA\_HOME).
- (Para macOS/Linux): A instalação pode ser feita via gerenciadores de pacotes (Homebrew: brew install maven, SDKMAN!: sdk install maven, APT: sudo apt install maven) ou manualmente, extraíndo o tar.gz e configurando M2\_HOME (opcional) e PATH no seu perfil do shell.



## Instalando uma IDE (IntelliJ IDEA / Eclipse)

- Contexto: Um Ambiente de Desenvolvimento Integrado (IDE) oferece recursos que aumentam muito a produtividade do desenvolvedor, como editor de código com autocompletar, depurador, integração com sistemas de controle de versão (como Git) e ferramentas de build (como Maven). Para desenvolvimento Java e Spring Boot, as opções mais populares e gratuitas são:
- IntelliJ IDEA Community Edition: (<https://www.jetbrains.com/idea/download/>) - Muito poderosa e com excelente suporte ao Spring Boot.
- Eclipse IDE for Java Developers (ou Spring Tools Suite - STS, baseado no Eclipse): (<https://www.eclipse.org/downloads/packages/>) - Outra IDE robusta e amplamente utilizada.

### Passos:

- 1) Download: Baixe o instalador para o seu sistema operacional a partir do site oficial da IDE escolhida.
- 2) Instalação: Execute o instalador e siga as instruções. A instalação geralmente é direta.
- 3) Configuração do JDK (se necessário): Após a instalação, abra a IDE. Ela pode detectar automaticamente o JDK instalado através da variável JAVA\_HOME. Caso contrário, você precisará configurar manualmente o caminho para o JDK nas configurações da IDE (geralmente em File > Project Structure > SDKs no IntelliJ ou Window > Preferences > Java > Installed JREs no Eclipse).

## Instalando o MySQL Community Server

Contexto: Precisamos de um Sistema de Gerenciamento de Banco de Dados (SGBD) para armazenar os dados das nossas tarefas de forma persistente. Utilizaremos o MySQL, um dos SGBDs relacionais mais populares do mundo. A versão Community é gratuita e ideal para aprendizado e desenvolvimento.

## Passos (Windows com MySQL Installer):

1. Download: Acesse a página de downloads do MySQL Community Server (<https://dev.mysql.com/downloads/mysql/>). É recomendado baixar o MySQL Installer for Windows. Ele facilita a instalação e configuração do servidor e outras ferramentas opcionais (como o MySQL Workbench).

### 2. Instalação: Execute o MySQL Installer.

- Choosing a Setup Type: Selecione "Developer Default" (instala servidor, Workbench, conectores, etc.) ou "Server only" se quiser apenas o servidor. Para este guia, "Server only" é suficiente, mas "Developer Default" pode ser útil.
- Check Requirements: O instalador pode verificar se há dependências faltando (como Microsoft Visual C++ Redistributable) e oferecer para instalá-las.
- Installation: Confirme os produtos a serem instalados e clique em Execute.
- Configuration: Esta é a parte crucial.
- Type and Networking: Mantenha as configurações padrão (Standalone MySQL Server, Development Computer, porta 3306).
- Authentication Method: Use a opção recomendada (geralmente "Use Strong Password Encryption").
- Accounts and Roles: Defina uma senha forte para o usuário root. Anote essa senha, pois você precisará dela! Não crie usuários adicionais aqui, faremos isso via linha de comando.
- Windows Service: Configure o MySQL para iniciar como um serviço do Windows (recomendado), mantendo o nome padrão (MySQLXX, onde XX é a versão).
- Apply Configuration: Clique em Execute para aplicar as configurações.
- Installation Complete: Finalize a instalação.

## Verificação:

- ✓ Serviço Windows: Pesquise por "Serviços" (services.msc) na barra de pesquisa do Windows.
- ✓ Procure pelo serviço do MySQL (ex: MySQL80) e verifique se o status é "Em Execução".
- ✓ (Opcional) MySQL Workbench: Se você instalou o Workbench, abra-o e tente conectar-se ao servidor local (localhost, porta 3306) usando o usuário root e a senha que você definiu.
- ✓ (Para macOS/Linux): A instalação pode ser feita via gerenciadores de pacotes (Homebrew: `brew install mysql`, APT: `sudo apt install mysql-server`, YUM: `sudo yum install mysql-server`) ou baixando o pacote .dmg (macOS) ou .deb/.rpm (Linux) do site oficial. Após a instalação, pode ser necessário iniciar o serviço (`sudo systemctl start mysql` ou `sudo service mysql start`) e executar o script de segurança (`sudo mysql_secure_installation`) para definir a senha do root e aplicar configurações de segurança recomendadas.

## Criando o Banco de Dados (tarefas\_db) e Usuário (tarefas\_user)

- ✓ Contexto: Para nossa aplicação Spring Boot, não é uma boa prática conectar-se ao banco de dados usando o usuário root, que tem privilégios administrativos totais. Em vez disso, criaremos um banco de dados dedicado para nossa aplicação (tarefas\_db) e um usuário específico (tarefas\_user) com permissões apenas para acessar esse banco de dados. Isso segue o Princípio do Menor Privilégio, uma prática de segurança importante.
- ✓ Passos (Linha de Comando MySQL):
- ✓ Conectar ao MySQL como root: Abra o prompt de comando ou terminal e digite:  
`Bash mysql -u root -p`  
Pressione Enter e, quando solicitado, digite a senha do usuário root que você definiu durante a instalação do MySQL.
- ✓ Executar Comandos SQL: Uma vez conectado ao prompt `mysql>`, execute os seguintes comandos SQL, pressionando Enter após cada um. Substitua 'senha\_segura' por uma senha forte de sua escolha e anote-a.

### **Criar o Banco de Dados:**

- SQL

```
CREATE DATABASE tarefas_db;
```

Este comando cria o banco de dados onde as tabelas da nossa aplicação serão armazenadas.

### **Criar o Usuário:**

- SQL

```
CREATE USER 'tarefas_user'@'localhost' IDENTIFIED BY 'senha_segura';
```

Este comando cria um usuário chamado tarefas\_user que só pode se conectar a partir da máquina local (localhost) com a senha especificada.

### **Conceder Privilégios:**

- SQL

```
GRANT ALL PRIVILEGES ON tarefas_db. * TO 'tarefas_user'@'localhost';
```

- Este é um passo crucial. Ele concede ao tarefas\_user todas as permissões (criar tabela, inserir, selecionar, atualizar, deletar, etc.) apenas sobre todos os objetos (\*) dentro do banco de dados tarefas\_db. Note que alguns exemplos podem mostrar GRANT ALL PRIVILEGES ON \*.\* , o que daria permissão total em *todos* os bancos de dados do servidor – isso é inseguro e deve ser evitado em ambientes reais. Para aprendizado, ALL PRIVILEGES no banco específico é aceitável.

### **Aplicar as Mudanças de Privilégios:**

- SQL FLUSH PRIVILEGES;
- Este comando recarrega as tabelas de privilégios, garantindo que as permissões recém-concedidas entrem em vigor imediatamente.

### Sair do MySQL:

- SQL EXIT;
- Verificação (Opcional): Você pode tentar conectar-se novamente ao MySQL usando o novo usuário para confirmar que ele foi criado e tem acesso (embora ainda não haja tabelas no banco tarefas\_db):
- Bash  
mysql -u tarefas\_user -p  
Digite a senha que você definiu para tarefas\_user. Se a conexão for bem-sucedida, o usuário e a senha estão corretos. Use EXIT; para sair.

**Tabela 1: Comandos SQL Essenciais para Configuração do Banco**

| Ação                 | Comando SQL                                                             |
|----------------------|-------------------------------------------------------------------------|
| Conectar como root   | mysql -u root -p                                                        |
| Criar Banco de Dados | CREATE DATABASE tarefas_db;                                             |
| Criar Usuário        | CREATE USER 'tarefas_user'@'localhost'<br>IDENTIFIED BY 'senha_segura'; |
| Conceder Privilégios | GRANT ALL PRIVILEGES ON tarefas_db.<br>TO 'tarefas_user'@'localhost';   |
| Aplicar Privilégios  | FLUSH PRIVILEGES;                                                       |
| Sair do MySQL        | EXIT;                                                                   |

## Criando o Projeto Spring Boot com Spring Initializr

- ✓ Com o ambiente pronto, vamos criar a estrutura básica do nosso projeto Spring Boot.
- ✓ Usaremos o Spring Initializr, uma ferramenta web que gera um projeto inicial com as configurações e dependências que especificarmos.

### Acessando start.spring.io

- ✓ Abra seu navegador e acesse <https://start.spring.io>. Você verá uma interface para configurar seu projeto.

### Selecionando Configurações e Dependências

#### – Preencha os campos da seguinte forma:

- Project: Selecione Maven Project.
- Language: Selecione Java.
- Spring Boot: Escolha uma versão estável recente (evite as que terminam em SNAPSHOT ou M). A versão padrão sugerida geralmente é uma boa escolha.
- Project Metadata:
- Group: Geralmente o domínio reverso da sua organização ou um identificador de grupo. Ex: com.example ou br.com.seudominio.
- Artifact: O nome do seu projeto (sem espaços). Ex: gerenciador-tarefas.
- Name: O mesmo que o Artifact (pode ser alterado).
- Description: Uma breve descrição do projeto. Ex: API para Gerenciamento de Tarefas.
- Package name: O pacote Java base para seu código. Será gerado a partir do Group e Artifact. Ex: com.example.gerenciadortarefas.
- Packaging: Selecione Jar. Aplicações Spring Boot são frequentemente empacotadas como JARs executáveis que contêm um servidor web embutido (como o Tomcat).
- Java: Selecione a versão do JDK que você instalou (ex: 17 ou 21).
- Dependencies: Clique no botão "ADD DEPENDENCIES..." ou use o campo de busca à direita. Adicione as seguintes dependências:

- Spring Web: Essencial para construir aplicações web e APIs RESTful usando Spring MVC e Tomcat embutido.
- Spring Data JPA: Facilita a interação com bancos de dados relacionais usando o padrão Java Persistence API (JPA). Ele simplifica a criação de camadas de acesso a dados (Repositories).
- MySQL Driver: O driver JDBC específico que permite à sua aplicação Java conectar-se e comunicar-se com um banco de dados MySQL.

## **Gerando e Importando o Projeto na IDE**

- Generate: Após configurar tudo, clique no botão "GENERATE" na parte inferior da página. Isso fará o download de um arquivo .zip contendo a estrutura do projeto Maven.
- Extract: Extraia o conteúdo do arquivo .zip para um local adequado em seu computador onde você guarda seus projetos.

## **Import: Abra sua IDE (IntelliJ IDEA ou Eclipse):**

- IntelliJ IDEA: Vá em File > Open... (ou Import Project em versões mais antigas) e navegue até a pasta que você acabou de extrair. Selecione o arquivo pom.xml ou a própria pasta e siga as instruções para abrir/importar como um projeto Maven.
- Eclipse: Vá em File > Import.... Na janela de importação, expanda a pasta Maven, selecione Existing Maven Projects e clique em Next. Clique em Browse... e selecione a pasta onde você extraiu o projeto. O projeto (com seu pom.xml) deve aparecer na lista. Certifique-se de que esteja marcado e clique em Finish.

## Maven Sync:

- A IDE detectará que é um projeto Maven e começará a baixar as dependências especificadas no arquivo pom.xml (Spring Web, Spring Data JPA, MySQL Driver etc.). Aguarde a conclusão desse processo (pode levar alguns minutos na primeira vez). Você verá o progresso geralmente na barra de status da IDE.

## Configurando a Conexão com o MySQL

- Agora que temos o projeto na IDE, precisamos informar ao Spring Boot como se conectar ao banco de dados tarefas\_db que criamos anteriormente, usando o usuário tarefas\_user.

## O arquivo application.properties

- Dentro da estrutura do seu projeto na IDE, navegue até a pasta src/main/resources. Lá você encontrará o arquivo application.properties.
- Este arquivo é o local padrão onde o Spring Boot procura por configurações da aplicação. Podemos definir várias propriedades aqui, como a porta do servidor, configurações de log e, crucialmente, os detalhes da conexão com o banco de dados. Como alternativa, pode-se usar o formato YAML (application.yml), que alguns desenvolvedores acham mais legível para configurações complexas. Para este guia, usaremos application.properties.

## Propriedades de DataSource (URL, Username, Password, Driver)

- Abra o arquivo application.properties e adicione as seguintes linhas, substituindo 'senha\_segura' pela senha que você definiu para o usuário tarefas\_user:

Properties

```
# Configuração da Conexão com o Banco de Dados MySQL
spring.datasource.url=jdbc:mysql://localhost:3306/tarefas_db?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true
spring.datasource.username=tarefas_user
spring.datasource.password=senha_segura
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```



## Explicação:

- `spring.datasource.url`: A URL de conexão JDBC.
- `jdbc:mysql://`: Indica que usaremos o driver JDBC para MySQL.
- `localhost:3306`: O endereço e a porta onde o servidor MySQL está rodando.
- `/tarefas_db`: O nome do banco de dados ao qual queremos nos conectar.
- `?useSSL=false`: Desabilita a exigência de conexão SSL (comum em ambientes de desenvolvimento local).
- `&serverTimezone=UTC`: Especifica o fuso horário do servidor para evitar problemas com datas e horas. Use o fuso horário apropriado se necessário.
- `&allowPublicKeyRetrieval=true`: Pode ser necessário para algumas versões recentes do driver MySQL ao usar autenticação baseada em senha mais segura.
- `spring.datasource.username`: O nome de usuário que criamos para a aplicação (`tarefas_user`).
- `spring.datasource.password`: A senha definida para as `tarefas_user`.
- `spring.datasource.driver-class-name`: O nome completo da classe do driver JDBC do MySQL. `com.mysql.cj.jdbc.Driver` é o driver moderno.

## Configuração do Hibernate (DDL-Auto)

- O Spring Data JPA utiliza o Hibernate por baixo dos panos como a implementação padrão da especificação JPA. Podemos configurar como o Hibernate interage com o esquema (estrutura das tabelas) do nosso banco de dados.
- Adicione as seguintes propriedades ao `application.properties`:  
Properties
- # Configuração do JPA e Hibernate  
`spring.jpa.hibernate.ddl-auto=update`  
`spring.jpa.show-sql=true`  
`# spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect`

## Explicação:

- `spring.jpa.hibernate.ddl-auto=update`: Esta é uma propriedade muito importante durante o desenvolvimento.
- `update`: instrui o Hibernate a comparar as definições das suas classes de entidade (que criaremos a seguir) com as tabelas existentes no banco de dados. Se uma tabela definida na entidade não existir, o Hibernate a criará. Se a tabela existir, mas faltarem colunas ou houver outras diferenças, o Hibernate tentará atualizá-la para corresponder à entidade, preservando os dados existentes. Cuidado: Em cenários complexos, a atualização automática pode falhar ou ter efeitos indesejados.

## Outras opções úteis:

- `validate`: Apenas verifica se o esquema do banco corresponde às entidades. Lança um erro na inicialização se houver diferenças. Útil para garantir consistência.
- `create`: Apaga todas as tabelas existentes das entidades gerenciadas e as recria do zero a cada inicialização da aplicação. Perde todos os dados! Útil para testes iniciais ou quando se quer um banco limpo a cada execução.
- `create-drop`: Similar ao `create`, mas também apaga as tabelas quando a aplicação é encerrada. Útil para testes automatizados.
- `none`: O Hibernate não faz nenhuma alteração no esquema do banco. Esta é a opção mais segura e recomendada para ambientes de produção, onde as alterações no banco de dados (migrações) devem ser gerenciadas por ferramentas específicas (como Flyway ou Liquibase).
- Para esta atividade, `update` é uma escolha conveniente.
- `spring.jpa.show-sql=true`: Faz com que o Hibernate imprima no console da aplicação todos os comandos SQL que ele gera e executa.<sup>21</sup> Isso é extremamente útil para depuração e para entender como o JPA está traduzindo suas operações em Java para SQL.
- `spring.jpa.properties.hibernate.dialect`: (Comentado) Especifica o "dialeto" SQL que o Hibernate deve usar para se comunicar com o banco de dados específico (MySQL neste caso). Geralmente, o Spring Boot consegue detectar o dialeto correto

automaticamente a partir da URL de conexão e do driver, então raramente é necessário definir isso explicitamente.

**Tabela 2: Configuração Essencial application. properties**

| Propriedade                         | Exemplo de Valor                                                                                    | Descrição                                                             |
|-------------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| spring.datasource.url               | jdbc:mysql://localhost:3306/tarefas_db?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true | URL de conexão JDBC para o banco de dados MySQL tarefas_db.           |
| spring.datasource.username          | tarefas_user                                                                                        | Nome de usuário para conectar ao banco.                               |
| spring.datasource.password          | senha_segura                                                                                        | Senha para o usuário tarefas_user.                                    |
| spring.datasource.driver-class-name | com.mysql.cj.jdbc.Driver                                                                            | Classe do driver JDBC do MySQL.                                       |
| spring.jpa.hibernate.ddl-auto       | update                                                                                              | Estratégia de geração/atualização do esquema do banco pelo Hibernate. |
| spring.jpa.show-sql                 | true                                                                                                | Exibe no console os comandos SQL gerados                              |

|  |  |                                   |
|--|--|-----------------------------------|
|  |  | pelo Hibernate (útil para debug). |
|--|--|-----------------------------------|

## Construindo as Camadas da Aplicação

- Com a configuração pronta, vamos criar as peças fundamentais da nossa aplicação: a Entidade (que representa nossos dados), o Repositório (que acessa o banco de dados) e o Controlador (que expõe nossa API). Em aplicações maiores, geralmente existe uma camada de Serviço (@Service) entre o Controlador e o Repositório para encapsular a lógica de negócio, mas para simplificar este primeiro exemplo, nosso Controlador interagirá diretamente com o Repositório.

### A Entidade (Tarefa.java com anotações JPA)

- Objetivo: Criar uma classe Java que funcione como um "molde" para os dados de uma tarefa e que o JPA possa mapear para uma linha na tabela correspondente no banco de dados MySQL.
- Código: Crie uma nova classe Java chamada Tarefa dentro do pacote base do seu projeto (ex: com.example.gerenciadortarefas) ou em um subpacote (ex: com.example.gerenciadortarefas.model ou com.example.gerenciadortarefas.entity).

#### Java

```
package com.example.gerenciadortarefas.entity; // Ou o pacote que você escolheu
```

```
import jakarta.persistence.Entity;  
import jakarta.persistence.GeneratedValue;  
import jakarta.persistence.GenerationType;  
import jakarta.persistence.Id;  
import jakarta.persistence.Table;  
import jakarta.persistence.Column;
```

```
@Entity // Marca esta classe como uma entidade JPA
```

```
@Table(name = "tarefas") // Mapeia para a tabela "tarefas" no banco (opcional se
```

nome da classe = nome da tabela)

```
public class Tarefa {
```

```
    @Id // Marca este campo como a chave primária
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Configura auto-  
    incremento (MySQL)
```

```
    private Long id;
```

```
    @Column(nullable = false) // Mapeia para a coluna 'descricao', não pode ser nula  
    private String descricao;
```

```
    @Column(nullable = false) // Mapeia para a coluna 'concluida', não pode ser nula  
    private boolean concluida;
```

```
    // Construtor padrão exigido pelo JPA
```

```
    protected Tarefa() {  
    }
```

```
    // Construtor para facilitar a criação de novas tarefas (sem ID, concluida=false por  
    padrão)
```

```
    public Tarefa(String descricao) {  
        this.descricao = descricao;  
        this.concluida = false; // Tarefas começam como não concluídas  
    }
```

```
    // Getters e Setters (necessários para JPA/Hibernate e outras bibliotecas)
```

```
    public Long getId() {  
        return id;  
    }
```

```
    public void setId(Long id) {  
        this.id = id;  
    }
```

```
public String getDescricao() {  
    return descricao;  
}  
  
public void setDescricao(String descricao) {  
    this.descricao = descricao;  
}  
  
public boolean isConcluida() {  
    return concluida;  
}  
  
public void setConcluida(boolean concluida) {  
    this.concluida = concluida;  
}  
  
// (Opcional) Método toString para facilitar a visualização em logs  
@Override  
public String toString() {  
    return "Tarefa{" +  
        "id=" + id +  
        ", descricao=" + descricao + "\" +  
        ", concluida=" + concluida +  
        "\"";  
}  
}
```

## Explicação das Anotações JPA (do pacote jakarta.persistence):

- **@Entity**: Informa ao JPA que esta classe é uma entidade e deve ser mapeada para uma tabela no banco de dados.
- **@Table(name = "tarefas")**: Especifica explicitamente o nome da tabela no banco. Se omitido, o JPA usaria o nome da classe (ex: Tarefa). É uma boa prática definir explicitamente.
- **@Id**: Designa o campo id como a chave primária da tabela. Cada linha na tabela terá um valor único neste campo.
- **@GeneratedValue(strategy = GenerationType.IDENTITY)**: Configura como o valor da chave primária (id) será gerado.
- **GenerationType.IDENTITY**: Delega a geração do ID para a coluna de auto-incremento do próprio banco de dados. É a estratégia mais comum e recomendada para MySQL.
- Outras estratégias incluem **AUTO** (o JPA escolhe a melhor estratégia), **SEQUENCE** (usa sequências do banco, comum em Oracle/PostgreSQL) e **TABLE** (usa uma tabela auxiliar para gerar IDs, mais portátil, mas menos performática).
- **@Column(nullable = false)**: Permite customizar o mapeamento de um campo para uma coluna. Aqui, estamos especificando que as colunas descricao e concluida não podem aceitar valores nulos (NOT NULL em SQL). Se **@Column** for omitido, o JPA mapeia o campo para uma coluna com o mesmo nome do campo.

## Repositório (TarefaRepository.java extendendo JpaRepository)

- **Objetivo**: Criar uma interface que define como nossa aplicação acessará os dados das tarefas no banco. O Spring Data JPA fará a "mágica" de implementar essa interface para nós.
- **Código**: Crie uma nova interface Java chamada TarefaRepository em um subpacote apropriado (ex: com.example.gerenciadortarefas.repository).

## Java

**package com.example.gerenciadortarefas.repository; // Ou o pacote que você escolheu**

**import com.example.gerenciadortarefas.entity.Tarefa;**

**import org.springframework.data.jpa.repository.JpaRepository;**

**import org.springframework.stereotype.Repository;**

**import java.util.List;**

**@Repository // Indica que esta interface é um componente de Repositório Spring (opcional, mas boa prática)**

**public interface TarefaRepository extends JpaRepository<Tarefa, Long> {**

**// Spring Data JPA implementará automaticamente os métodos CRUD básicos:  
// save(), findById(), findAll(), deleteById(), etc.**

**// Exemplo de método de consulta derivado (Query Method):**

**// Encontra todas as tarefas com um determinado status de conclusão.**

**// O Spring Data JPA gera a consulta SQL correspondente baseada no nome do método.**

**List<Tarefa> findByConcluida(boolean concluida);**

**// Você pode adicionar outros métodos de consulta conforme necessário.**

**// Ex: List<Tarefa> findByDescricaoContainingIgnoreCase(String keyword);**

**}**



## Explicação:

- @Repository: Anotação do Spring que marca a interface como um componente de acesso a dados (DAO/Repository). Embora não seja estritamente necessária quando se estende JpaRepository em versões mais recentes do Spring Boot (pois a extensão já a torna elegível para detecção), é uma boa prática para clareza.
- public interface TarefaRepository extends JpaRepository<Tarefa, Long>: Esta é a linha principal.
- Declaramos uma interface pública TarefaRepository.
- Ela extends JpaRepository: Isso significa que nossa interface herda um conjunto completo de métodos para manipulação de dados da entidade especificada.<sup>18</sup> JpaRepository por sua vez estende PagingAndSortingRepository e CrudRepository, fornecendo funcionalidades CRUD, paginação e ordenação.
- <Tarefa, Long>: Especificamos os tipos genéricos:
- Tarefa: A classe da entidade que este repositório gerenciará.
- Long: O tipo da chave primária (@Id) da entidade Tarefa.
- A Magia do Spring Data JPA: O ponto mais impressionante aqui é que não precisamos escrever nenhuma implementação para os métodos CRUD básicos (save, findById, findAll, deleteById, count, existsById, etc.). Ao estender JpaRepository, o Spring Data JPA automaticamente cria uma implementação funcional em tempo de execução. Isso elimina uma quantidade enorme de código repetitivo (boilerplate) que seria necessário em abordagens mais antigas como JDBC puro ou mesmo JPA puro sem Spring Data.
- List<Tarefa> findByConcluida (boolean concluida);: Este é um exemplo de *método de consulta derivado* (query method). Apenas declarando a assinatura do método seguindo as convenções de nomenclatura do Spring Data JPA (findBy<NomeDoCampo>), o framework automaticamente entende que queremos buscar tarefas filtrando pelo campo concluida e gera a consulta SQL apropriada (SELECT... FROM tarefas WHERE concluida =?).<sup>18</sup> Existem muitas outras palavras-chave que podem ser usadas para criar consultas mais complexas ((And Or, Containing, IgnoreCase, GreaterThan, etc.).

## O Controlador REST (TarefaController.java com endpoints CRUD)

- Objetivo: Criar a classe que atuará como a porta de entrada da nossa API. Ela receberá as requisições HTTP vindas dos clientes (como o Postman ou um navegador), usará o TarefaRepository para realizar as operações no banco de dados e enviará as respostas HTTP de volta.
- Código: Crie uma nova classe Java chamada TarefaController em um subpacote apropriado (ex: com.example.gerenciadortarefas.controller).

### Java

```
package com.example.gerenciadortarefas.controller; // Ou o pacote que você escolheu
```

```
import com.example.gerenciadortarefas.entity.Tarefa;  
import com.example.gerenciadortarefas.repository.TarefaRepository;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.http.HttpStatus;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;
```

```
import java.util.List;  
import java.util.Optional;
```

```
@RestController // Combina @Controller e @ResponseBody, indicando que os  
retornos dos métodos são os dados da resposta  
@RequestMapping("/api/tarefas") // Define o prefixo de URL base para todos os  
endpoints neste controlador  
public class TarefaController {
```

```
    private final TarefaRepository tarefaRepository;
```

```
    // Injeção de Dependência via Construtor (recomendado)
```

```
    @Autowired
```

```
    public TarefaController(TarefaRepository tarefaRepository) {
```

```
this.tarefaRepository = tarefaRepository;
}

// Endpoint para CRIAR uma nova tarefa (POST /api/tarefas)
@PostMapping
public ResponseEntity<Tarefa> criarTarefa(@RequestBody Tarefa tarefa) {
    // Garante que a tarefa nova sempre comece como não concluída
    tarefa.setConcluida(false);
    // Garante que não estamos tentando definir um ID manualmente na criação
    tarefa.setId(null);
    Tarefa novaTarefa = tarefaRepository.save(tarefa);
    return new ResponseEntity<>(novaTarefa, HttpStatus.CREATED); // Retorna 201
Created
}

// Endpoint para LISTAR TODAS as tarefas (GET /api/tarefas)
@GetMapping
public ResponseEntity<List<Tarefa>> listarTarefas() {
    List<Tarefa> tarefas = tarefaRepository.findAll();
    return new ResponseEntity<>(tarefas, HttpStatus.OK); // Retorna 200 OK
}

// Endpoint para BUSCAR uma tarefa por ID (GET /api/tarefas/{id})
@GetMapping("/{id}")
public ResponseEntity<Tarefa> buscarTarefaPorId(@PathVariable Long id) {
    Optional<Tarefa> tarefaOptional = tarefaRepository.findById(id);

    // Verifica se a tarefa foi encontrada
    if (tarefaOptional.isPresent()) {
        return new ResponseEntity<>(tarefaOptional.get(), HttpStatus.OK); // Retorna
200 OK com a tarefa
    } else {
        return new ResponseEntity<>(HttpStatus.NOT_FOUND); // Retorna 404 Not
Found
    }
}
```

```
}  
  
// Alternativa mais concisa usando lambda (Java 8+):  
// return tarefaRepository.findById(id)  
//     .map(tarefa -> new ResponseEntity<>(tarefa, HttpStatus.OK))  
//     .orElse(new ResponseEntity<>(HttpStatus.NOT_FOUND));  
}  
  
// Endpoint para ATUALIZAR uma tarefa existente (PUT /api/tarefas/{id})  
@PutMapping("/{id}")  
public ResponseEntity<Tarefa> atualizarTarefa(@PathVariable Long id,  
@RequestBody Tarefa tarefaAtualizada) {  
    Optional<Tarefa> tarefaExistenteOptional = tarefaRepository.findById(id);  
  
    if (tarefaExistenteOptional.isPresent()) {  
        Tarefa tarefaExistente = tarefaExistenteOptional.get();  
        // Atualiza os campos da tarefa existente com os dados recebidos  
        tarefaExistente.setDescricao(tarefaAtualizada.getDescricao());  
        tarefaExistente.setConcluida(tarefaAtualizada.isConcluida());  
        // Salva a tarefa atualizada no banco  
        Tarefa tarefaSalva = tarefaRepository.save(tarefaExistente);  
        return new ResponseEntity<>(tarefaSalva, HttpStatus.OK); // Retorna 200 OK  
com a tarefa atualizada  
    } else {  
        return new ResponseEntity<>(HttpStatus.NOT_FOUND); // Retorna 404 Not  
Found  
    }  
}  
  
// Endpoint para DELETAR uma tarefa por ID (DELETE /api/tarefas/{id})  
@DeleteMapping("/{id}")  
public ResponseEntity<Void> deletarTarefa(@PathVariable Long id) {  
    // Verifica se a tarefa existe antes de tentar deletar  
    if (tarefaRepository.existsById(id)) {  
        tarefaRepository.deleteById(id);  
    }  
}
```

```
return new ResponseEntity<>(HttpStatus.NO_CONTENT); // Retorna 204 No
Content (sucesso, sem corpo)
    } else {
        return new ResponseEntity<>(HttpStatus.NOT_FOUND); // Retorna 404 Not
Found
    }
}
```

## Explicação:

- **@RestController**: Define a classe como um controlador REST. O Spring MVC automaticamente serializará os objetos retornados pelos métodos para JSON (ou outro formato, dependendo da configuração e dos cabeçalhos Accept da requisição) e os colocará no corpo da resposta HTTP.
- **@RequestMapping("/api/tarefas")**: Mapeia todas as requisições que começam com /api/tarefas para os métodos dentro desta classe.<sup>29</sup> Usar um prefixo como /api é comum para APIs.
- **Injeção de Dependência**:
- **private final TarefaRepository tarefaRepository;** declara uma dependência da nossa interface de repositório.
- **@Autowired public TarefaController (TarefaRepository tarefaRepository) {...}**: Este é o construtor da classe. A anotação **@Autowired** instrui o Spring a injetar (fornecer) automaticamente uma instância concreta de TarefaRepository (criada pelo Spring Data JPA) quando criar uma instância de TarefaController. Esta é a forma recomendada de fazer injeção de dependência (injeção via construtor).
- **Métodos de Endpoint**: Cada método público na classe corresponde a uma operação da nossa API e é anotado com uma anotação de mapeamento HTTP específica:
- **@PostMapping**: Mapeia requisições HTTP POST para /api/tarefas. Usado para criar recursos.
- **@GetMapping**: Mapeia requisições HTTP GET para /api/tarefas (listar todos) ou /api/tarefas/{id} (buscar um).<sup>1</sup> Usado para recuperar recursos.
- **@PutMapping("/{id}")**: Mapeia requisições HTTP PUT para /api/tarefas/{id}.<sup>29</sup> Usado para atualizar um recurso existente completamente.

- `@DeleteMapping("/{id}")`: Mapeia requisições HTTP DELETE para `/api/tarefas/{id}`.<sup>29</sup> Usado para remover um recurso.
- Anotações de Parâmetros:
- `@RequestBody` Tarefa tarefa: Indica que o parâmetro tarefa deve ser preenchido com os dados desserializados do corpo da requisição HTTP (geralmente JSON).<sup>29</sup> Usado em POST e PUT.
- `@PathVariable` Long id: Indica que o parâmetro id deve ser extraído da variável de caminho na URL (o valor que substitui `{id}`). Usado em GET por ID, PUT e DELETE.
- Uso do Repositório: Os métodos do controlador utilizam a instância injetada `tarefaRepository` para interagir com o banco de dados, chamando os métodos fornecidos pelo Spring Data JPA (`save`, `findAll`, `findById`, `existsById`, `deleteById`).
- `ResponseEntity`: Em vez de retornar diretamente a entidade ou uma lista, os métodos retornam `ResponseEntity<?>`. Isso nos dá mais controle sobre a resposta HTTP, permitindo definir explicitamente o status HTTP (ex: `HttpStatus.CREATED`, `HttpStatus.OK`, `HttpStatus.NOT_FOUND`, `HttpStatus.NO_CONTENT`) além do corpo da resposta. É uma prática recomendada para APIs REST.

## Testando a API com Postman

- Com o código pronto, precisamos executar nossa aplicação Spring Boot e testar se os endpoints da API estão funcionando como esperado. O Postman é uma excelente ferramenta gráfica para isso.

## Executando a Aplicação Spring Boot

- Na sua IDE, localize a classe principal da aplicação (aquela com a anotação `@SpringBootApplication` e o método `main`, geralmente chamada `GerenciadorTarefasApplication.java` ou similar, criada pelo Spring Initializr).
- Clique com o botão direito sobre essa classe e selecione "Run" ou "Debug".
- Observe o console da IDE. Você deverá ver logs do Spring Boot iniciando. Se tudo estiver configurado corretamente (JDK, Maven, `application.properties`), você verá mensagens indicando que o Tomcat iniciou na porta 8080 (ou a porta que você configurou) e que a aplicação está pronta.<sup>1</sup> Se houver erros, verifique os logs cuidadosamente (erros de conexão com o banco, problemas de compilação, etc.).

## Configurando o Postman

- Se ainda não o fez, baixe e instale o Postman (<https://www.postman.com/downloads/>).
- Abra o Postman. Para organizar nossos testes, crie uma nova "Collection": clique em "New" > "Collection", dê um nome (ex: "API Tarefas Spring Boot") e clique em "Create".

## Testando o Endpoint POST /api/tarefas (Criar)

- 1) Dentro da sua collection, clique em "Add Request". Dê um nome (ex: "Criar Tarefa").
- 2) Método: Selecione POST no dropdown.
- 3) URL: Digite `http://localhost:8080/api/tarefas`. (Verifique se a porta 8080 está correta).

### Body:

- Clique na aba "Body".
- Selecione a opção raw.
- No dropdown à direita que aparece, selecione JSON.
- Na área de texto, insira o JSON para a nova tarefa:

JSON

```
{  
  "descricao": "Aprender a testar APIs com Postman"  
}
```

(Note que não enviamos id nem concluída, pois serão definidos pelo backend).

- 1) Headers: Clique na aba "Headers". Verifique se há um cabeçalho Content-Type com o valor `application/json`. O Postman geralmente adiciona isso automaticamente quando você seleciona raw e JSON no Body.

### 2) Send: Clique no botão

1. "Send".

### 3) Verificação:

- Status: No painel de resposta (Response), verifique se o Status é 201 Created.
- Body: Verifique se o corpo da resposta contém o JSON da tarefa criada, agora com um id (ex: 1) e concluída como false.

JSON

```
{  
  "id": 1,
```

"descricao": "Aprender a testar APIs com Postman",

"concluida": false

}



- (Opcional) Teste Postman: Clique na aba "Tests" da *requisição* (não da resposta). Adicione o seguinte script para verificar o status:

- JavaScript

```
pm.test("Status code is 201 Created", function () {
```

```
    pm.response.to.have.status(201);
```

```
});
```

Clique em "Send" novamente. A aba "Test Results" na *resposta* deve mostrar "PASS".

## Testando o Endpoint GET /api/tarefas (Listar Todas)

- 1) Crie uma nova request ("Listar Tarefas").
- 2) Método: GET.
- 3) URL: `http://localhost:8080/api/tarefas`.
- 4) Body: Nenhuma (não aplicável para GET padrão).
- 5) Send: Clique em "Send".
- 6) Verificação:

- Status: 200 OK.

- Body: Deve ser um array JSON contendo a tarefa (ou tarefas) que você criou anteriormente. Exemplo:

### JSON

```
{
```

```
    "id": 1,
```

```
    "descricao": "Aprender a testar APIs com Postman",
```

```
    "concluida": false
```

```
}
```



- Teste Postman: Adicione um teste para verificar o status 200:

#### JavaScript

```
pm.test("Status code is 200 OK", function () {  
    pm.response.to.have.status(200);  
});
```

### Testando o Endpoint GET /api/tarefas/{id} (Buscar por ID)

- Crie uma nova request ("Buscar Tarefa por ID").
- Método: GET.
- URL: `http://localhost:8080/api/tarefas/1` (Substitua 1 pelo ID da tarefa que você criou).
- Send: Clique em "Send".
- Verificação (ID existente):
- Status: 200 OK.
- Body: Deve conter o JSON da tarefa com o ID 1.

#### Verificação (ID inexistente):

- Altere a URL para um ID que não existe, ex: `http://localhost:8080/api/tarefas/999`.
- Clique em "Send".
- Status: 404 Not Found.
- Body: Geralmente vazio ou com uma mensagem de erro padrão do Spring Boot.
- Teste Postman: Você pode adicionar testes para ambos os cenários.

### Testando o Endpoint PUT /api/tarefas/{id} (Atualizar)

1. Crie uma nova request ("Atualizar Tarefa").
2. Método: PUT.
3. URL: `http://localhost:8080/api/tarefas/1` (Use o ID da tarefa existente).
4. Body: Selecione raw e JSON. Insira o JSON com os dados atualizados:

#### JSON

```
{  
    "descricao": "Tarefa atualizada via Postman!",
```

**"concluida": true**

**}**

- Headers: Verifique Content-Type: application/json.
  - Send: Clique em "Send".
  - Verificação:
- Status: 200 OK.
  - Body: Deve conter a tarefa com os dados atualizados (descricao e concluida modificados).
  - Confirmação: Execute novamente a request "Buscar Tarefa por ID" para o ID 1 e verifique se os dados retornados refletem a atualização.
  - Verificação (ID inexistente): Tente atualizar um ID inexistente (ex: 999). O Status esperado é 404 Not Found.

## **Testando o Endpoint DELETE /api/tarefas/{id} (Deletar)**

1. Crie uma nova request ("Deletar Tarefa").
2. Método: DELETE.
3. URL: http://localhost:8080/api/tarefas/1 (Use o ID da tarefa existente).
4. Send: Clique em "Send".
5. Verificação:
  - Status: 204 No Content (ou 200 OK, dependendo da implementação do controller). 204 é semanticamente mais correto para DELETE bem-sucedido sem retorno de corpo.
  - Body: Vazio (para status 204).
  - Confirmação: Execute a request "Listar Tarefas" novamente. A tarefa com ID 1 não deve mais aparecer na lista. Execute "Buscar Tarefa por ID" para o ID 1; deve retornar 404 Not Found.
  - Verificação (ID inexistente): Tente deletar um ID inexistente (ex: 999). O Status esperado é 404 Not Found.

**Tabela 3: Resumo dos Testes no Postman**

| Operação<br>CRUD | Método<br>HTTP | URL Exemplo       | Exemplo de<br>Corpo<br>(Body)                           | Status<br>Esperado<br>(Sucesso) | Status<br>Esperado<br>(ID Não<br>Encontra<br>do) |
|------------------|----------------|-------------------|---------------------------------------------------------|---------------------------------|--------------------------------------------------|
| Criar            | POST           | /api/tarefas      | {"descricao":<br>"Nova<br>Tarefa"}                      | 201<br>Created                  | N/A                                              |
| Listar<br>Todas  | GET            | /api/tarefas      | Nenhum                                                  | 200 OK                          | N/A                                              |
| Buscar por<br>ID | GET            | /api/tarefas/{id} | Nenhum                                                  | 200 OK                          | 404 Not<br>Found                                 |
| Atualizar        | PUT            | /api/tarefas/{id} | {"descricao":<br>"Atualizada",<br>"concluida":<br>true} | 200 OK                          | 404 Not<br>Found                                 |
| Deletar          | DELETE         | /api/tarefas/{id} | Nenhum                                                  | 204 No<br>Content /<br>200 OK   | 404 Not<br>Found                                 |

## **(Opcional) Testando a API com curl**

- Para aqueles que preferem a linha de comando ou precisam automatizar testes em scripts, a ferramenta curl é uma alternativa poderosa ao Postman.
- Contexto: curl é uma ferramenta de linha de comando para transferir dados com URLs, suportando diversos protocolos, incluindo HTTP. É amplamente disponível em sistemas Linux e macOS, e pode ser instalada no Windows.
- Exemplos de Comandos: Abra seu terminal ou prompt de comando e execute os seguintes comandos, ajustando IDs e dados conforme necessário (lembre-se que no Windows, pode ser necessário usar aspas duplas " externamente e escapar as aspas internas \" para o JSON no comando -d):

### **Criar Tarefa (POST):**

#### **•Bash**

- ✓ curl -X POST -H "Content-Type: application/json" -d '{"descricao":"Tarefa criada via curl", "concluida":false}' http://localhost:8080/api/tarefas
- ✓ -X POST: Especifica o método HTTP POST.<sup>34</sup>
- ✓ -H "Content-Type: application/json": Define o cabeçalho Content-Type.
- ✓ -d "...": Envia os dados JSON no corpo da requisição.
- ✓ Listar Todas as Tarefas (GET): (GET é o padrão, -X GET é opcional)  
Bash curl <http://localhost:8080/api/tarefas>

### **(Adicione -v para ver detalhes da requisição/resposta).**

- ✓ Buscar Tarefa por ID (GET): (Substitua 2 pelo ID desejado)  
Bash curl <http://localhost:8080/api/tarefas/2>

### **Atualizar Tarefa (PUT): (Substitua 2 pelo ID desejado)**

- ✓ Bash
- ✓ curl -X PUT -H "Content-Type: application/json" -d '{"descricao":"Tarefa atualizada via curl", "concluida":true}' http://localhost:8080/api/tarefas/2



Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

## **-X PUT: Especifica o método HTTP PUT.**

- ✓ Deletar Tarefa (DELETE): (Substitua 2 pelo ID desejado)

Bash curl -X DELETE http://localhost:8080/api/tarefas/2

## **-X DELETE: Especifica o método HTTP DELETE.**

- ✓ Analisar a saída no terminal após cada comando para verificar o resultado (corpo da resposta e, se usar -v, os cabeçalhos e status HTTP).
- ✓ Conclusão e Próximos Passos
- ✓ Parabéns! Ao seguir os passos desta Situação de Aprendizagem, vocês construíram e testaram com sucesso sua primeira aplicação web utilizando Spring Boot, Spring Data JPA e MySQL.